Hello everyone,

This is the first chapter of my dissertation (if it's not obvious from the name). Feel free to review as much as you would like. I am finished with the introduction for the most part and am currently working on the methods and case study part. I've made external outlines for each paragraph to show what I would like to communicate. I am looking for feedback on structure and clarity, and I have one comment about what I should prioritize in this paper; should I structure it in the order of use (start with the buffering methods and then introduce the point extractions) or should I prioritize the point extractions with the buffering modules as a supplemental package feature.

Thank you!

Title: *skiba*: a forester's package to perform point extractions from Google Earth Engine

Authors: Tara Skiba, Tyler Gifford, Thomas Brandeis

Journal of interest: Forestry (Oxford Academic)

Citable Nuggets

1. Remotely sensed data can be easier to incorporate into forest studies and inventories if technological barriers are removed
2. Existing python packages can be combined to develop a more efficient workflow to extract pixel values from remotely sensed data in GEE.
3. Due to concerns about running sensitive coordinates through GEE, two buffering approaches were developed to use in conjunction with the point extraction tools.

Abstract

Introduction

Forests, which cover 31% of the Earth's land area, are complex ecosystems with high variability and therefore challenging to study (Wei *et al.* 2025). A forest's characteristics, such as species composition, development patterns, etc, are influenced not only by the previous forest ecosystems that preceded the current forest, but also the environmental conditions, like temperature, rainfall, or sunlight intervals (Toledo *et al.* 2011, Itter *et al.* 2017). While environmental factors are important to understanding forest systems, they are hard to generate and not consistently included in operational inventories. For example, metrics like average rainfall or temperature is calculated from long-term continuous monitoring equipment (Thammanu *et al.* 2020). Not only would this be difficult to obtain for small forest studies, but it also becomes impossible to quantify at large spatial scales, like national forest inventories (NFIs). Without this information provided, understanding about how environmental variables influence forest systems is limited. These information gaps have been filled using geospatial data for the following reasons: it is spatially uninterrupted, may be continuously monitored, and provides metrics that are best captured via remotely sensed methods like satellite or multi-

Commented [ed1]: which is more important? more widely used? sounds like the point extractions, so I would lead with that

Commented [NM2]: Very clear, I like them

Commented [ed3]: i'm not sure I caught the two

Commented [TS4]: 1.Forests take up a lot of space on Earth.
2.Forest ecosystems are varied, and influenced by previous forests and current environmental conditions
3.Environmental data is hard to produce in forest inventories, especially at large scales.
4.Why is environmental data important?
5.RS data can provide this information/has been used to fill these information gaps.
6.Because this data is spatially continuous and constantly measured, we can use this information to complement inventories over time and any spatial scale.
7.This helps us with estimation better than ground-based inv alone.

Commented [ed5]: You need a topic sentence (TS) that gets you from forests to remote sensing. Something like: Forests are complex, highly variable systems whose structure and dynamics are shaped by environmental conditions that are often difficult to measure at meaningful spatial scales.

Commented [PK6]: I think there are two different ideas in one sentence here, which makes it a little hard to follow - 1. forests take up a lot of space and 2. they are complex with high variability, which makes them hard to study

Commented [PK7]: what do you mean by factors are hard to generate?

Commented [PK8]: this feels like a jargony term, and I'm not quite sure what it means, but could def be a field difference!!!

Commented [BE9]: Not quite sure I'm following the path of information in these sentences. Maybe more explanation?

Commented [PK10]: what information are you referring to here?

Commented [PK11]: you use this and these a lot, and it makes it a little hard to follow. Maybe consider adding some concrete nouns in so that readers can follow your line of thought!

spectral band imagery *et al.* 2024, Jha *et al.* 2024). These valuable characteristics can be exploited to complement plot-based forest inventories to enable estimation of forest health, canopy structure, biomass, disturbance, and climate drivers at scales impossible through ground-based inventories alone.

Despite the plethora of existing geospatial data available, the specialized technical knowledge needed to utilize geospatial technologies poses a significant barrier for foresters to overcome, thereby preventing geospatial data from being used to its fullest potential (Ziegler and Chasins 2023, Fassnacht *et al.* 2024). Existing geospatial processing technologies are powerful but still require geospatial literacy and programming skills. Geospatial software like ArcGIS eliminates the majority of the programming knowledge needed through its program interface which makes it a popular software within natural resources, but users must still have geospatial data knowledge to perform any operations (Morgenroth and Visser 2013). Open-source programs like Python and R provide the most user flexibility with geospatial data handling but are more technically advanced due to the programming required. In addition, licenses may have to be purchased, which can be expensive and deter potential users. Even if users have the required technical knowledge and software available, other challenges remain. One major obstacle is the decentralization of data, as geospatial datasets are collected and maintained by numerous government agencies, both domestically and internationally. These datasets are hosts among the various agencies' websites, which can pose issues not only with locating the desired dataset, but also with downloading these incredibly large datasets. These challenges can make geospatial data retrieval, handling, and processing inefficient.

A solution to the issue of data decentralization can be found when using Google Earth Engine (GEE), a free, public repository of many popular remotely sensed and other geospatial datasets (Mutanga and Kumar 2019). These datasets are stored in a cloud server, which allow users to avoid having to download the large datasets locally, increasing efficiency. GEE also has a Code Editor where users can work completely on a cloud server and employs the JavaScript language (Amani *et al.* 2020, Ermida *et al.* 2020). For non-JavasScript users, GEE has been integrated into existing geospatial analysis software. The GEE Toolbox provides GEE integration in ArcGIS products (Google Earth Engine Community [2024] 2026). The R package *rgee* provides GEE integration into the R environment by translating R code to Python and passing it to the Google Earth Engine API (Aybar *et al.* 2020). Meanwhile, earthengine-*api* (ee) in Python provides users the full processing flexibility similar to GEE's Code Editor, and Python packages like *geemap* provides additional functions that streamline the processing workflow (Wu 2020, Google Earth Engine, n.d.). Such tools provide easier access to GEE and reduce some of the legwork needed to perform geospatial analysis, but the technical threshold for manipulating geospatial data remains.

One way for RS data to be paired with operational forest inventories is to perform simple point extractions of the pixel value that contains an inventory plot's coordinates. To accomplish

Commented [LM12]: This is a really nice start — the logic is clear and I can see the flow you're aiming for. I think a few small tweaks (especially tightening the intro and simplifying some sentences) could make it even stronger and more focused on the main message about data limitations and RS filling that gap.

Commented [TS13]: 1.There is lots of RS data available, but required technical knowledge prevents it from being used. Existing programs are powerful, but complex to use
a. Python and R: flexible but technically advanced
b. ArcGIS: point-and-click user interface but still need geospatial knowledge. Expensive license.
2.Existing RS data is decentralized across government agency websites
3.Downloading data is very large

Commented [PK14]: I feel like this transition could be improved - are geospatial technologies the same as ground-based inventories? if not, it's a quick jump to something new

Commented [ed15R14]: agree but I do like that the TS matches the info that is in this paragraph.

Commented [LM16]: This paragraph clearly communicates the main barriers, and the progression of ideas is easy to follow. You might consider tightening some of the software examples (ArcGIS vs. Python/R) to keep the focus on the broader challenge rather than the details. Also, the final sentence could be sharpened slightly to more directly emphasize how these barriers limit the practical use of geospatial data.

Commented [TS17]: 1.GEE is a solution to data decentralization.
2.Also, since it's a cloud server, data does not need to be downloaded locally
3.Packages and Toolboxes already exist that integrate GEE into the geospatial software and programs describ ...

Commented [LM18]: This paragraph does a nice job introducing GEE as a solution and outlining available tools, but you might streamline some of the technical details to ...

Commented [TS19]: 1.What do we need to do to use RS data to complement forest inventories: conduct point extractions.
2.What does the current process look like? ...

Commented [PK20]: did you define RS somewhere above before abbreviating? I might've missed it!!

Commented [ed21R20]: I always say avoid any abbreviations

Commented [ed22]: The TS provides a road map - you don't have to hide your point at the end of the paragraph. Something like: A common approach for integrating remotely sensed (RS) data with forest inventories is point- ...

this, users would have to upload their coordinates, access GEE and select a dataset, perform any necessary data manipulations to ensure object (file, format) compatibility, extract the desired information, and return the results. While these point extractions are possible to produce with existing software, there is a need to make this process more efficient and less technically complicated. This entire workflow can be distilled down to only require the user to provide the necessary input information and perform all other steps behind the scenes. The creation of a streamlined workflow would not only be beneficial to foresters lacking the technical knowledge to access RS data, but it would also allow experienced geospatial data users to perform simple point extractions quickly and efficiently.

Because forest inventories can contain highly detailed information of public and private lands, which can also be used to estimate the monetary value of the existing timber, actual plot coordinates may be protected under confidentiality agreements to preserve plot integrity and protect private landowners' properties. These confidentiality agreements may not allow data users to run these protected coordinates through GEE without modification. The U.S.'s national forest inventory, which is maintained by the USDA U.S. Forest Service's Forest Inventory and Analysis program, provides publicly available coordinates that are buffered using the methodology described in the *General Technical Report WO* 2005. Proprietary or other government forest inventories may not have an established buffering methodology as these datasets are not publicly accessible. Users of these datasets may have concerns about using unprotected plot coordinates in GEE, given the public nature of Google services. To alleviate these concerns, a buffering approach that quickly creates randomized the raw plot coordinates within a given radius was developed. The newly created plot coordinates protect the plots' actual locations and can be safely run in GEE.

Currently, a streamlined workflow that allows forest practitioners to specifically perform point-extractions from GEE-derived raster data for inventory coordinates without advanced coding or proprietary software does not exist. In this study, we introduce *skiba* , a Python package that integrates existing GEE tools into a simplified workflow that minimizes coding requirements and lowers technical barriers. By pairing efficient point extraction with a flexible buffering approach for protected coordinates, this package allows users to access and retrieve remotely sensed data with fewer steps and less specialized expertise. This workflow expands the accessibility of GEE-based analyses and facilitates the integration of remotely sensed data into forest studies and inventories.

Methods

Software design

The *skiba* Python package was designed for users to reduce technical barriers for accessing Google Earth Engine and producing point extractions. The returned point extractions can be integrated into forest operations to increase the scope of information that can be used in existing forest inventories and studies. The *skiba* package addresses privacy concerns regarding

Commented [ed23]: This is a super long TS! Can we tighten? Confidentiality constraints on forest inventory data create practical challenges for integrating plot locations with publicly accessible geospatial tools. or maybe more specific 'particularly when relying on public platforms such as Google Earth Engine (GEE).

Commented [ed24]: This paragraph is straddling two tones - an introduction and a discussion. This mainly happens in the last couple of sentences. Perhaps instead use 'to alleviate these concerns, buffering approaches that create randomized raw plot coordinates within a given radius is needed..' Such an approach would enable the use of protected plot data in GEE without compromising location confidentiality.

Commented [TS25]: 1.Gap where this specific program does not yet exist.
2.We created this package to remove the technical barriers and make a more efficient process to conduct point extractions.
3.Due to issues with running sensitive plot coordinates through GEE, we also created two buffering methods to randomize coordinates provided.
4.The two groups of modules allow users to safely and quickly extract data from GEE.

Commented [AO26]: I like the final paragraph here ending with a gap

Commented [TS27]: 1.This package is designed to remove technical barriers
2.The returned point extraction data can be used in this way.
3.Confidentiality concerns are addressed via buffering methods
4.We used point-and-click widgets to lower the technical threshold needed to perform these operations.
5.This design handles/streamlines these processes.

sensitive plot coordinates by providing two buffering methodologies. To lower the technical threshold needed to perform all operations, we designed this package to create a user interface consisting of point-and-click widgets. This design interface handles file uploading, coordinate processing, and file returns, so that package users need only to simply load the modules and provide the information needed. The workflows created, while designed to remove technical barriers involved, can also provide efficient alternatives to existing approaches through decreased set-up time and quick processing.

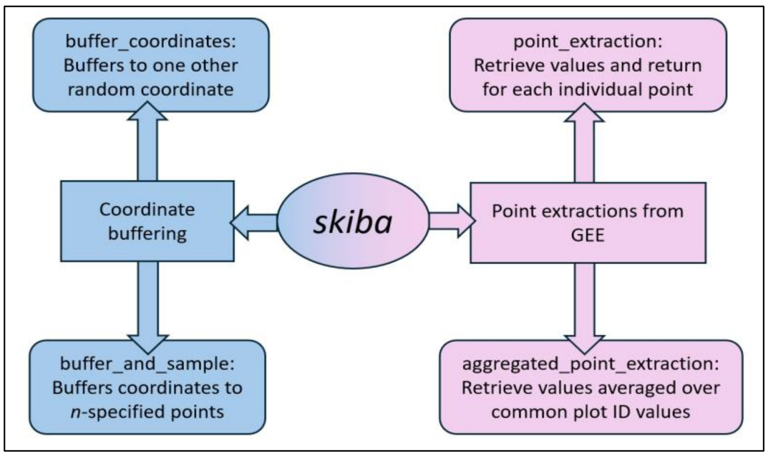


Figure 1: Diagram of package structure

The package is divided into two task groups: point extractions and coordinate buffering, which can be visualized in Figure 1. The point extraction approaches access and retrieve information from GEE, while the coordinate buffering approaches provide the option to safeguard confidential plot coordinates prior to point extraction. Each task group contains two modules, for a total of four created modules. This package relies on a number of dependencies. The *ipywidgets* package serves as the foundation for all modules to construct the visible(?) user interface (Martindale *et al.* 2024). Pandas and *geopandas* are also leveraged in each module to handle geospatial file handling. The *ee* and *geemap* Python packages are exclusively called within the point extraction modules to handle Google Earth Engine objects and retrieve pixel values. In the buffering approaches, *shapely*, *shapely.geometry*, *pyproj*, and *numpy* help generate random coordinate points. As these packages do not access GEE, any coordinate points run through these buffering modules are not seen by Google.

Commented [TS28]: 1.How this package is divided up into two groups
2.Point extraction vs coordinate buffering, with two modules each.
3.This package relies on a number of dependencies.
Ipywidgets is the foundation
Packages used in all 4 modules
Point extraction only packages (significant because it is not called in the buffering approaches to protect sensitive plot locations)
Buffering approach packages

Commented [TS29]: I've been going back and forth on which task group to introduce first.
Is the focus of the package the point extractions, so it should be first?
Should I introduce them in the chronological order of use? (buffer first, then run point extractions)
- Update figures to be consistent in order of final choice.

Point Extraction Modules

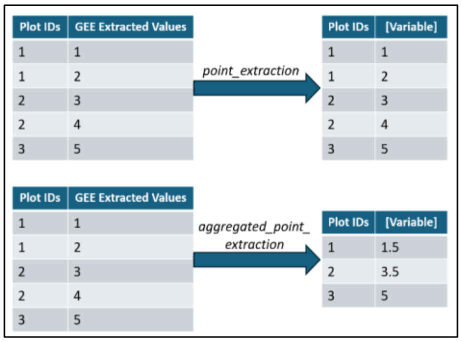
The point extraction modules provide a widget-oriented approach to performing pixel extractions from GEE datasets. Because these modules are designed with a user interface comprised of point-and-click widgets, once widget is imported and loaded, no additional coding is required; the package user will only need to provide a datafile of coordinate points, select the desired GEE dataset and optional time filter, if the latter is necessary. As it accesses Google Earth Engine internally, GEE must be initialized and authenticated prior to running the query.

The point extraction modules require that GEE be initialized and authenticated prior to pixel extraction. When the modules' interfaces are loaded(?), a widget box is shown for users to upload a CSV file of coordinate points is uploaded, select the GEE dataset of interest, and set filter dates. Acceptable GEE datasets must Images and ImageCollections, not FeatureCollections. Start and end dates to filter that datasets are required only for ImageCollections and may be left blank for Images. The user may refer to the dataset's GEE documentation page to find dataset availability. A link to the GEE dataset's documentation page is conveniently provided within the widget once a dataset is selected.

Two point-extraction modules were created: *point_extraction* (PE) and *aggregated_point_extraction* (APE). Both modules perform the same extraction process but differ in aggregation technique. PE returns the extracted data for each coordinate uploaded, whereas APE extracts the pixel values then averages over shared plot IDs. The difference between returned results is demonstrated in Figure 4.

The following script is used to initialize GEE and load the point extraction modules:

```
import ee
# Initialize Earth Engine
ee.Authenticate()
ee.Initialize(project="[your project name]")
# For non-aggregated point extraction
import skiba.point_extraction as spe
point = spe.PointExtraction().vbox
point
# For aggregated point extraction
import skiba.aggregated_point_extraction as sape
agg = sape.AggregatedPointExtraction().vbox
agg
```



Plot IDs	GEE Extracted Values
1	1
1	2
2	3
2	4
3	5

point_extraction

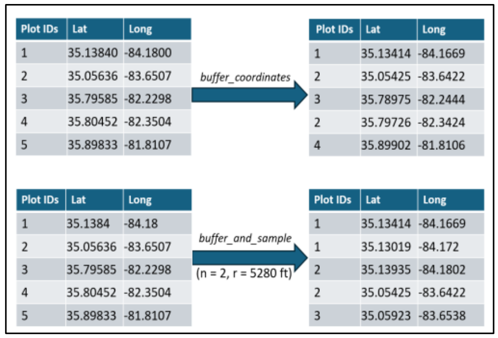
Plot IDs	[Variable]
1	1
1	2
2	3
2	4
3	5

Plot IDs	GEE Extracted Values
1	1
1	2
2	3
2	4
3	5

aggregated_point_extraction

Plot IDs	[Variable]
1	1.5
2	3.5
3	5

Figure 2: Demonstration of how data aggregations differ for the two point-extraction modules.



Plot IDs	Lat	Long
1	35.13840	-84.1800
2	35.05636	-83.6507
3	35.79585	-82.2298
4	35.80452	-82.3504
5	35.89833	-81.8107

buffer_coordinates

Plot IDs	Lat	Long
1	35.13414	-84.1669
2	35.05425	-83.6422
3	35.78975	-82.2444
2	35.79726	-82.3424
4	35.89902	-81.8106

Plot IDs	Lat	Long
1	35.1384	-84.18
2	35.05636	-83.6507
3	35.79585	-82.2298
4	35.80452	-82.3504
5	35.89833	-81.8107

buffer_and_sample

(n = 2, r = 5280 ft)

Plot IDs	Lat	Long
1	35.13414	-84.1669
1	35.13019	-84.172
2	35.13935	-84.1802
2	35.05425	-83.6422
3	35.05923	-83.6538

Figure 3: Demonstration of point creation differences for the two coordinate buffering modules.

Coordinate Buffering Modules

Users who have concerns about operating GEE using sensitive plot have the option to buffer coordinates prior to point extractions. The two buffering modules operate separately of the point extraction modules, and as it does not initialize GEE, coordinates are not passed through GEE. In the coordinate buffering modules, users provide a CSV file containing the raw coordinates and set the buffer radius for randomized point(s) to be generated within. This buffer radius r can be determined via the data's user guide, the pixel size for the GEE dataset of interest, and user discretion. Additionally, if the plot radius is considerably large, it may need to be considered when determining an appropriate buffer radius. The returned dataset is returned already formatted for use in point extraction modules with the original coordinates removed. As the returned datasets do not include original coordinates and GEE is not initialized in these

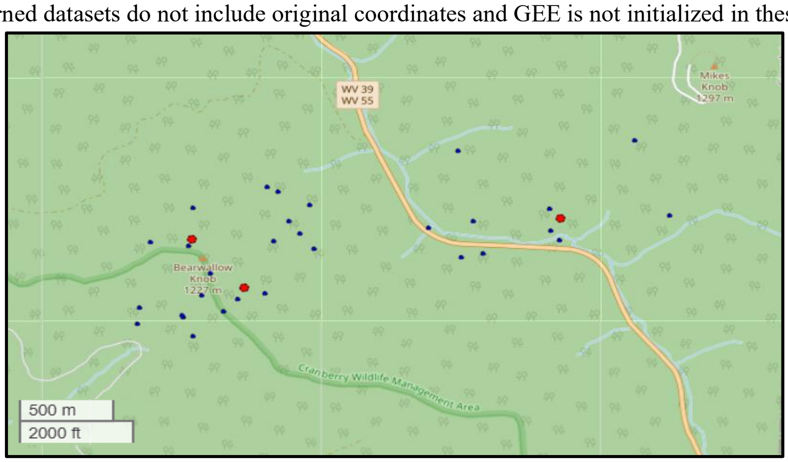


Figure 4: Example of buffer_and_sample approach with r = 3000 ft and n = 10. Uploaded coordinates are marked red, and the 10 buffered coordinates are blue.

Commented [BE30]: I think it makes sense to keep buffering modules second like you have now because not everyone will use that feature

Commented [AO31R30]: I agree since this is a chapter in your dissertation, where I feel like you have free reign to actually explore all the aspects of the project without worrying about what is "supplemental" like you said in your note. But as someone who has never written a dissertation before, I could be wrong.

modules, the original coordinates remain protected from GEE. Returned datasets are formatted for immediate use in the point extraction modules.

Two modules were created to buffer sensitive coordinates. Both modules follow the same randomization process; buffered coordinates are generated at a randomized angle and distance within *r*. The difference between the two modules lies in how many randomized coordinates are generated, seen in Figure 3. *buffer_coordinates* (BC) produces one randomized coordinate for each user-provided coordinate; *buffer_and_sample* (BAS) returns *n* random coordinates for each provided coordinate, where *n* is an integer provided by the user. A visualization of this buffering approach is provided in.

Using buffered coordinates to conduct point extractions could return biased results. Some visual examples of how buffered coordinates may lie outside of the original coordinate's pixel are shown in. This is demonstrated in Figure 4, where some buffered coordinates lie in a neighboring pixel. Users may desire to repeat the buffering process and point extractions to compare differences in returned values and determine if the values are acceptable.

The following script is used to load the point buffering modules:

```
# For single point buffering
import skiba.buffer_coordinates as sbc
single = sbc.buffer_coordinates().vbox
single
# For multiple point buffering
import skiba.buffer_and_sample as sbs
multiple = sbs.buffer().vbox
multiple
```

Input Data

The user-provided file of coordinate points to run through all developed modules must be properly formatted. This file must be saved as a CSV file and contain the three following columns: a plot identifier column, a latitude column, and a longitude column. Accepted naming conventions for these columns are shown in Table 1. The plot identifier column is used as a reference column, which is column that is grouped over in the point creation and averaging processes conducted in the BAS and APE modules. Latitude and longitude columns should be formatted in decimal degrees [later, crs is set to EPSG:4326, if relevant to add].

Commented [AO32]: This needs a transition before, maybe using "To input data..." otherwise this sentence is sort of lonely

Workflow

TBD

Methodological limitations

Table 1: Accepted column names for user-provided list of coordinates. File must be saved as a CSV.

CSV Columns	Accepted Names
latitude	lat, latitude, y, LAT, Latitude, Y
longitude	log, long, longitude, x, LON, Longitude, Long, X
plot ID	id, ID, plot_ID, plot_id, plotID, plotId

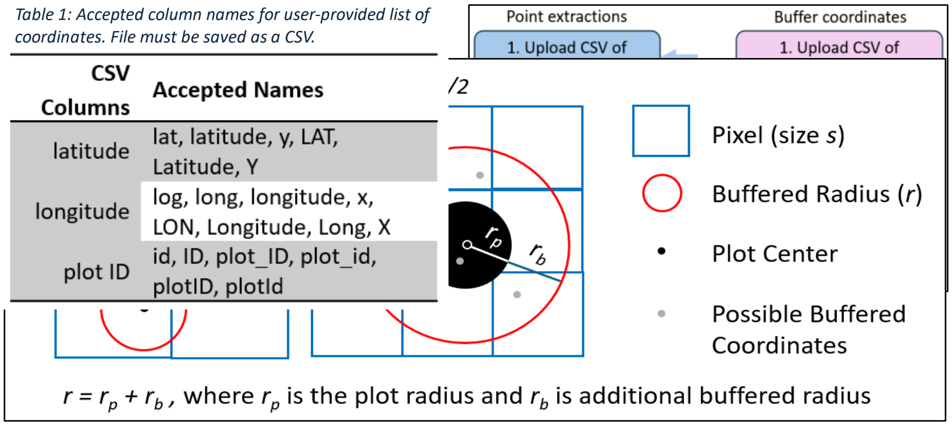


Figure 5: Examples of where randomized pixels may be created in pixels different from the original coordinate

Some limitations have been observed regarding dataset combability. For example, WorldClim Climatology V1 (WORLDCLIM/V1/MONTHLY) and NEON Canopy Height Model (projects/neon-prod-earthengine/assets/CHM/001) are both ImageCollections, which is able to be handled by the created modules. However, both of these datasets contain metadata fields attached to the collection objects themselves called Image Properties, which are used to filter, sort, and select images. This changes the process for filtering GEE datasets and was not accounted for/considered in the design process of the *skiba* package. Therefore, GEE ImageCollections that contain Image Properties are incompatible with this package.

Commented [AO33]: I think you need a sentence before this introducing limitations, since this just dives right in

Streamlit web app

While the *skiba* python package removes some technical barriers and streamlines coordinate buffering and point extraction processes, the technical threshold for Python can still be a significant hurdle. The set-up time can be a lengthy and challenging process that deters potential users.

Many statistical and geospatial programs are available to buffer coordinates and extract information from geospatial data, and inconsistencies in user expertise across the various technologies are well established. Because of this, users who want to use this package may not be proficient in Python who are not proficient in Python willwho wish to use this package could still limited by the knowledge required to operate in Python. The technical barriers for working in Python environments may still be too preventative for some to benefit from this package.

Removing all the skills needed to execute this packing in Python can be accomplished through developing a web application where the package is directly built into the site.

Commented [BE34]: Is this something you will be working on in the future? If so, could it be listed as "limitations and future research"? Just seems that be a more positive spin than talking about barriers that still exist (even though your tools makes it easier)

It removes the following:

The technical skills needed to set up and operate python

The need to create a GEE account to run these operations through.

Advantages:

Easily accessible from any computer (no need for a python environment)

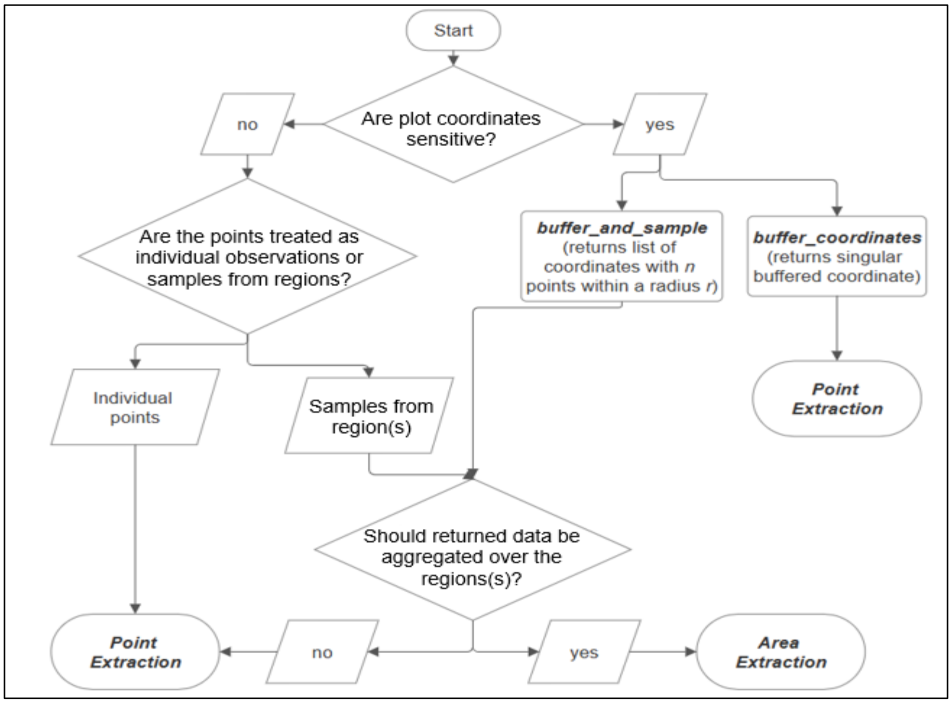


Figure 6: Decision tree to determine which module to select given input data and desired results.

Limitations:

Issues with debugging issues (not visible to non-maintainers)

Limited functionality and development (flexibility)

Case study – Forest Inventory

We demonstrate the utility of the *skiba* package with expanding existing forest inventories by querying environmental data to combine with the USDA U.S. Forest Service Forest Inventory and Analysis's (FIA) long term forest monitoring database. The FIA's forest plots are [description of plot size, tree level and stand level measurements, and what not.] Plot coordinates are designated as the plot center of the microplot. To preserve the integrity of this long-term monitoring program and the information of private landowners' properties included in the inventory, the exact plot coordinates are classified. However, the FIA provides publicly available plot coordinates that are buffered within a 1-mile radius. An additional swapping of plot identifiers adds an additional layer of protection for plots located on private lands. The full methodology for this process can be found in [] (Bechtold and Patterson 2015). For this case study, we obtained both the public and confidential plot coordinates to analyze against the buffering approaches designed. As only coordinates located within public lands of national forests and parks of southern Appalachia are used, the public plot coordinates are only buffered, not swapped.

The WorldClim BIO variables V1 dataset in GEE serves as the environmental dataset of interest. This dataset has been used extensively/broadly in ecological studies because it includes detailed precipitation and temperature information with 19 variables at a resolution of 927.67 meters (~1 km). This resolution size is compatible with the FIA plot inventory size (All 4 subplots:1/6 ac, 1 subplot: 1/24 ac. Plot: ~ 1 ac same as landsat)

The resolution size is larger than the FIA's microplot, but smaller than the distance public plot coordinates are from the actual inventory plots. Therefore, using the public plot coordinates alone to generate environmental data to complement the FIA inventory can lead to biased results.

2286 – when using raw data, 2906 for pub/buff

Conclusions

This package has an open-source repository on GitHub and is downloadable from PyPI using the following command:

```
# install skiba
pip install skiba
```

As this package is hosted on GitHub, users have the option to expand on the current applications by creating branches and creating new functions. This package establishes a user-friendly framework that can be further developed to perform additional functions that will benefit foresters and natural resource professionals.

References

Amani M, Ghorbanian A, Ahmadi SA *et al.* Google Earth Engine Cloud Computing Platform for Remote Sensing Big Data Applications: A Comprehensive Review. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing* 2020;**13**:5326–50. https://doi.org/10.1109/JSTARS.2020.3021052.

Aybar C, Wu Q, Bautista L *et al.* rgee: An R package for interacting with Google Earth Engine. *JOSS* 2020;**5**(51):2272. https://doi.org/10.21105/joss.02272.

Chi M, Plaza A, Benediktsson JA *et al.* Big Data for Remote Sensing: Challenges and Opportunities. *Proc IEEE* 2016;**104**(11):2207–19. https://doi.org/10.1109/JPROC.2016.2598228.

Chinembiri Tsikai S., Mutanga O, Dube T. A multi-source data approach to carbon stock prediction using Bayesian hierarchical geostatistical models in plantation forest ecosystems. *GIScience & Remote Sensing* 2024;**61**(1):2303868. https://doi.org/10.1080/15481603.2024.2303868.

Chinembiri Tsikai Solomon, Mutanga O, Dube T. Landsat-8 and Sentinel-2 Based Prediction of Forest Plantation C Stock Using Spatially Varying Coefficient Bayesian Hierarchical Models. *Remote Sensing* 2022;**14**(22):5676. https://doi.org/10.3390/rs14225676.

Ermida SL, Soares P, Mantas V *et al.* Google Earth Engine Open-Source Code for Land Surface Temperature Estimation from the Landsat Series. *Remote Sensing* 2020;**12**(9). https://www.mdpi.com/2072-4292/12/9/1471 (2 Apr. 2026, date last accessed).

Fassnacht FE, White JC, Wulder MA *et al.* Remote sensing in forestry: current challenges, considerations and directions. *Forestry: An International Journal of Forest Research* 2024;**97**(1):11–37. https://doi.org/10.1093/forestry/cpad024.

General Technical Report WO. n.p.: Forest Service, U.S. Department of Agriculture, 2005.

Google Earth Engine. n.d. https://earthengine.google.com (15 Sept. 2025, date last accessed).

Google Earth Engine Community. *Gee-Community/Arcgis-Earthengine-Toolbox*, Python. 30 July 2024, 6 Feb. 2026. https://github.com/gee-community/arcgis-earthengine-toolbox (2 Apr. 2026, date last accessed).

Itter MS, Finley AO, D'Amato AW *et al.* Variable effects of climate on forest growth in relation to climate extremes, disturbance, and forest dynamics. *Ecological Applications* 2017;**27**(4):1082–95. https://doi.org/10.1002/eap.1518.

Jha S, Yang SI, Hale DS *et al.* *Quantifying Forest Aboveground Biomass Using Remote Sensing and Ground Measurements for Mixed Hardwood Forests in East Tennessee*. 2024;**274**:51–7. https://doi.org/10.2737/SRS-GTR-274-Pap8.

Martindale N, Smith J, Linville L. *Supporting Greater Interactivity in the IPython Visualization Ecosystem - SciPy Proceedings*. 10 July 2024. https://proceedings.scipy.org/articles/GVHT1072 (1 Apr. 2026, date last accessed).

Morgenroth J, Visser R. Uptake and barriers to the use of geospatial technologies in forest management. *New Zealand Journal of Forestry Science* 2013;**43**(1):16. https://doi.org/10.1186/1179-5395-43-16.

Mutanga O, Kumar L. Google Earth Engine Applications. *Remote Sensing* 2019;**11**(5). https://www.mdpi.com/2072-4292/11/5/591 (2 Apr. 2026, date last accessed).

Thammanu S, Marod D, Han H *et al.* The influence of environmental factors on species composition and distribution in a community forest in Northern Thailand. *Journal of Forestry Research* 2020;**32**(2):649–62. https://doi.org/10.1007/s11676-020-01239-y.

Toledo M, Poorter L, Peña-Claros M *et al.* Climate is a stronger driver of tree and forest growth rates than soil and disturbance. *Journal of Ecology* 2011;**99**(1):254–64. https://doi.org/10.1111/j.1365-2745.2010.01741.x.

Wei X, Hayes D, McHale G *et al.* PyFIA: analyzing and visualizing forest attributes using the United States Forest Inventory and Analysis database. *Carbon Balance Manage* 2025;**21**(1):18. https://doi.org/10.1186/s13021-025-00364-7.

Wu Q. geemap: A Python package for interactive mapping with Google Earth Engine. *Journal of Open Source Software* 2020;**5**(51):2305. https://doi.org/10.21105/joss.02305.

Yang SI, Brandeis TJ, Helmer EH *et al.* Characterizing height-diameter relationships for Caribbean trees using mixed-effects random forest algorithm. *Forest Ecology and Management* 2022;**524**:120507. https://doi.org/10.1016/j.foreco.2022.120507.

Ziegler P, Chasins SE. A Need-Finding Study with Users of Geospatial Data. *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (New York, NY, USA) CHI '23, 19 Apr. 2023:1–16. https://doi.org/10.1145/3544548.3581370.

Random thoughts saved for later:

Remotely sensed data can also provide information that is collected via airborne or spaceborne vehicles like the Normalized Difference Vegetation Index (NDVI)

In addition to the daily measurement challenges, some metrics used to study forest systems like the Normalized Difference Vegetation Index (NDVI) are produced using expensive instruments that are commonly collected via airborne or spaceborne vehicles

As a python package, there is still coding knowledge needed to set up and install the required packages. To completely remove this technical barrier, a Streamlit web app was built for users to utilize this package without any prior local setup. This website authenticates GEE through a designated Google account to remove the need for the user to create their own.

Code font

Use active voice

For coherence, have the last part of a sentence be the same as the beginning of the next sentence.

Use first person a lot, passive voice judiciously

Strip the paper of the exact system (like removing birds, or the forest inventory part). What are you trying to get at? Extracting pixel values from GEE data in a non-technical way. (then it is in the context of forest inventories.)